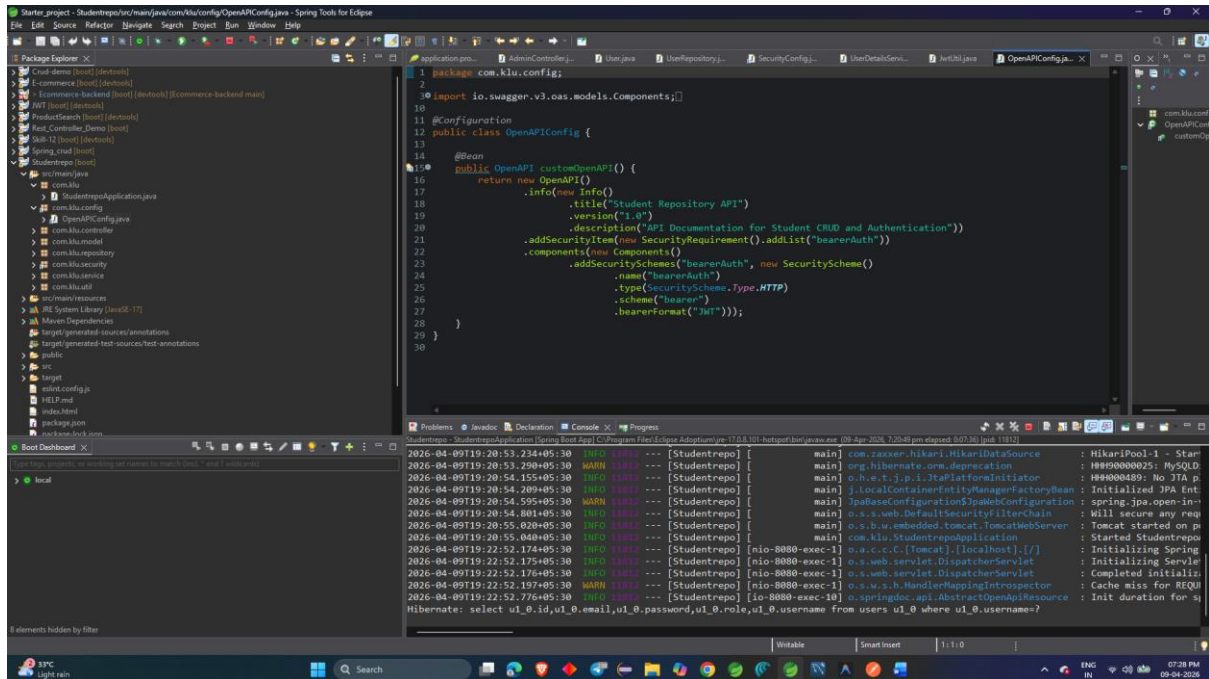
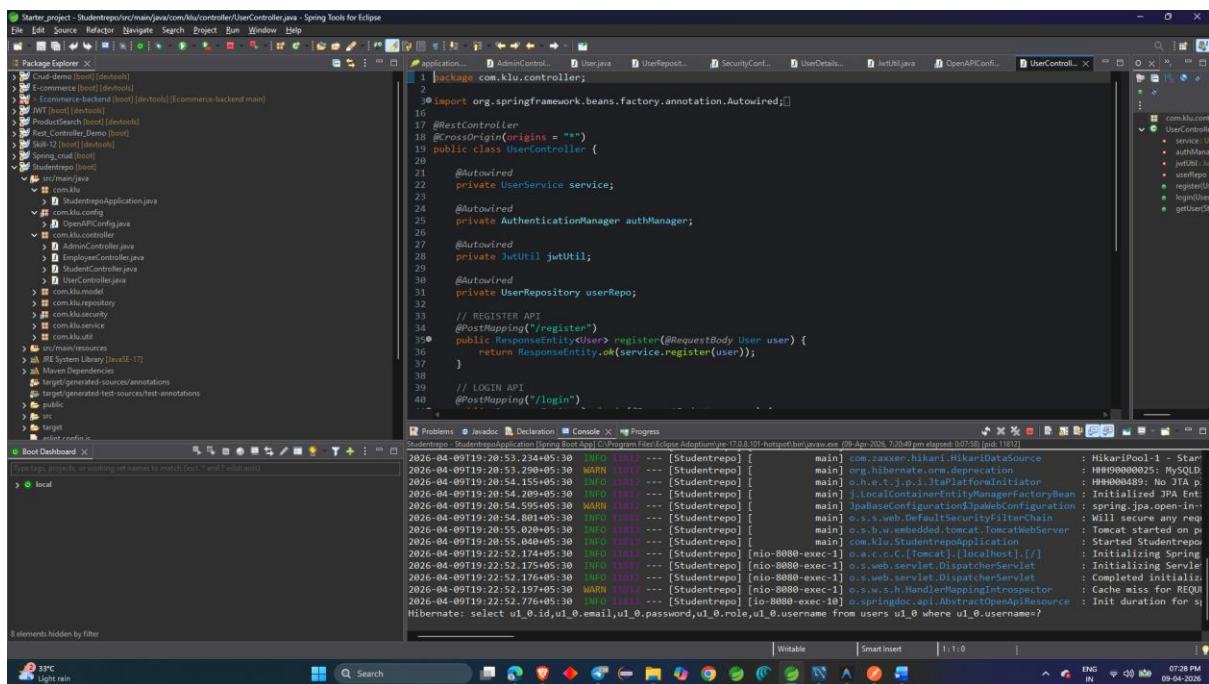




The screenshot shows the Eclipse IDE with the `OpenAPIConfig.java` file open. The file is part of the `com.klu` package and implements the `OpenAPI` interface. It includes Swagger annotations for API documentation and security configurations.

```
1 package com.klu.config;
2
3 import io.swagger.v3.oas.models.Components;
4
5 @Configuration
6 public class OpenAPIConfig {
7
8     @Bean
9     public OpenAPI customOpenAPI() {
10         return new OpenAPI()
11             .title("Student Repository API")
12             .version("1.0")
13             .description("API Documentation for Student CRUD and Authentication")
14             .addSecurityItem(new SecurityRequirement().addList("bearerAuth"))
15             .components(new Components()
16                 .addSecurityScheme("bearerAuth", new SecurityScheme()
17                     .name("bearerAuth")
18                     .type(SecurityScheme.Type.HTTP)
19                     .scheme("bearer")
20                     .bearerFormat("JWT"))));
21     }
22 }
```

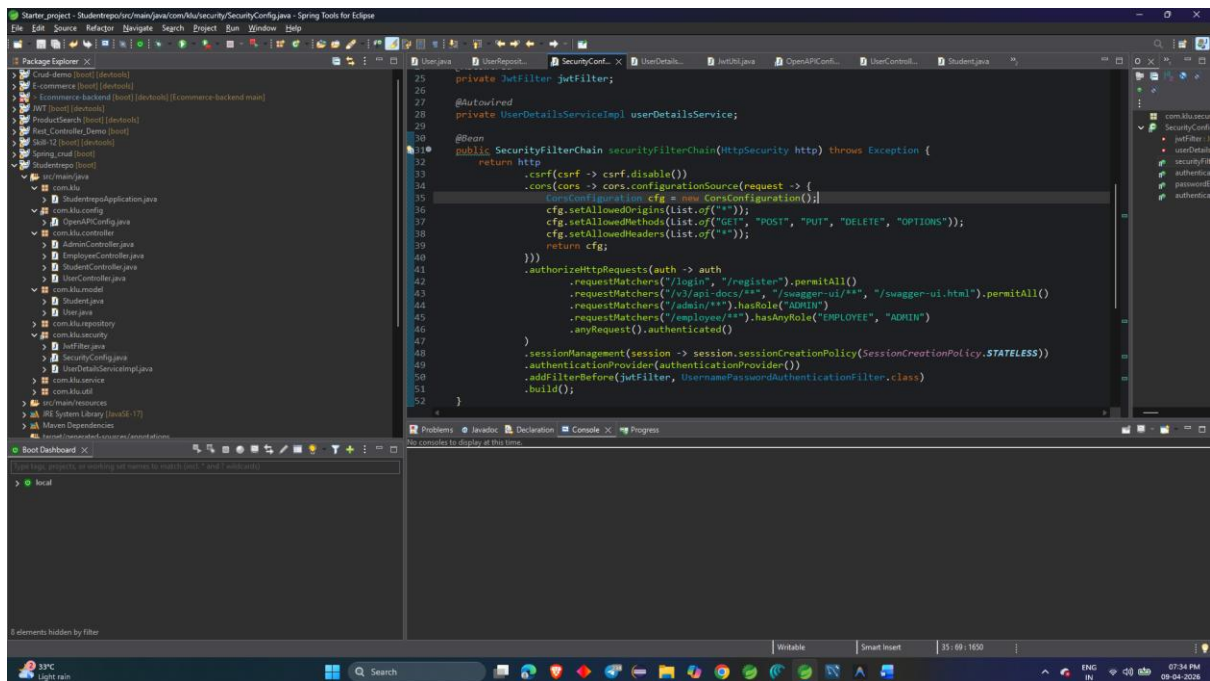
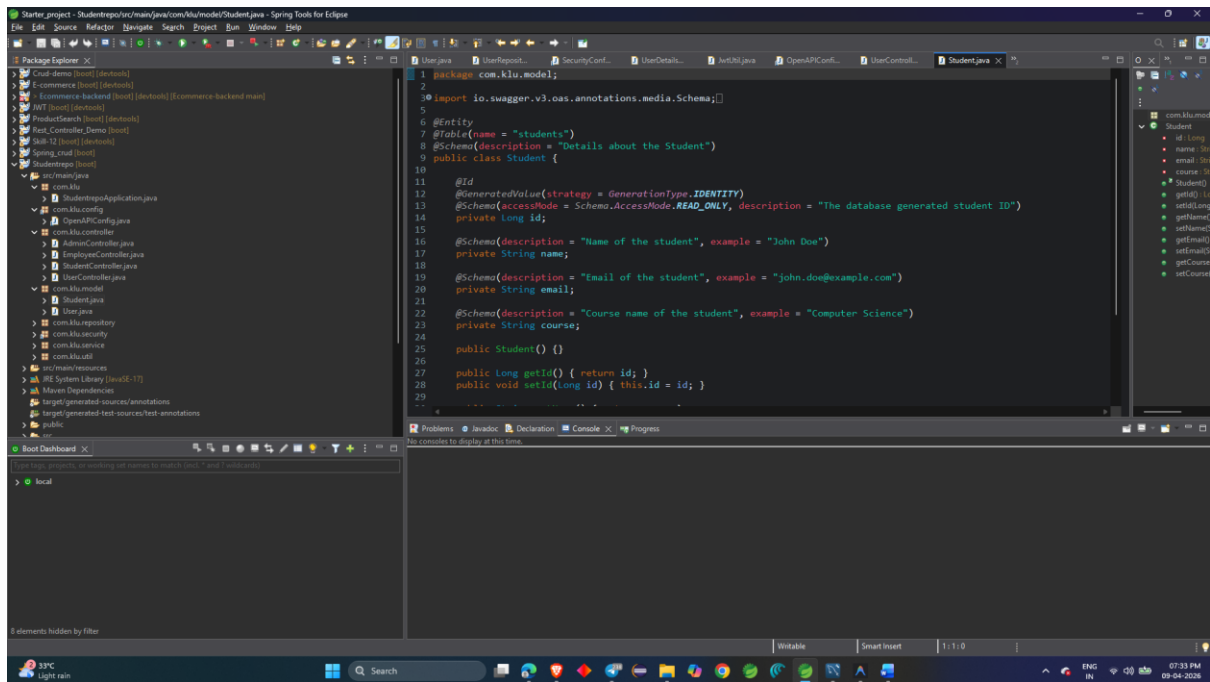
The console output shows the application startup logs, including the initialization of the HikariPool, JPA EntityManagerFactory, and the Spring application context.

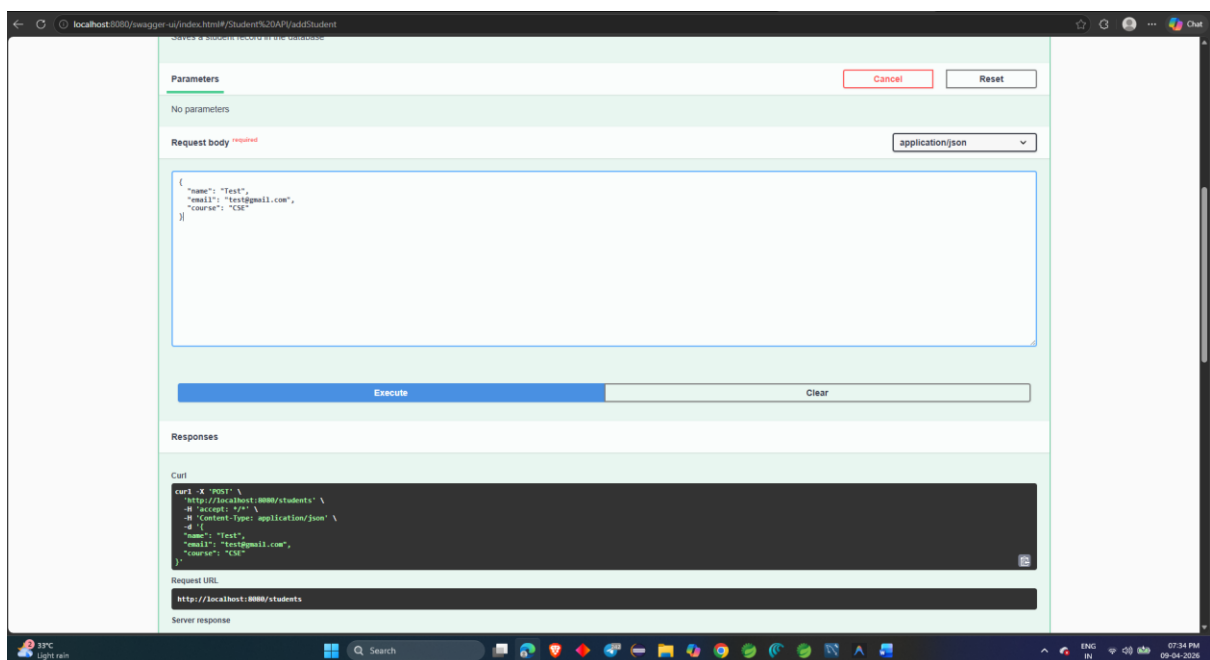
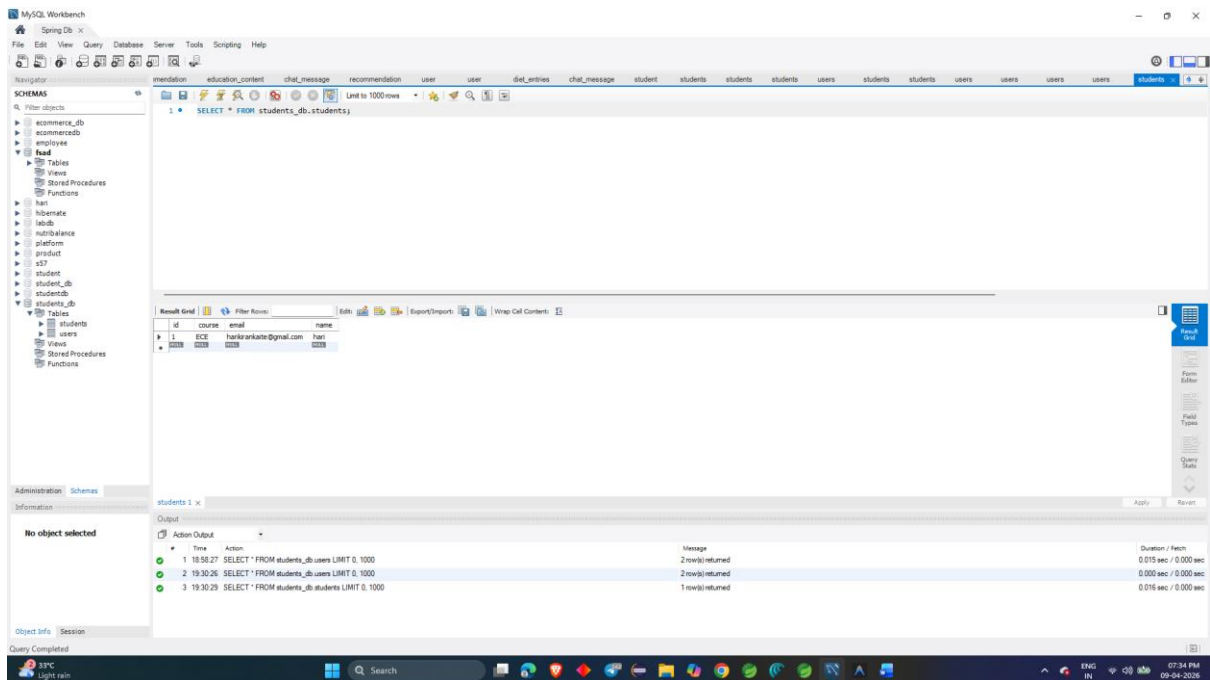


The screenshot shows the Eclipse IDE with the `UserController.java` file open. The file is part of the `com.klu` package and implements the `UserController` interface. It includes Spring annotations for REST API endpoints and dependency injection.

```
1 package com.klu.controller;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4
5 @RestController
6 @CrossOrigin(origins = "**")
7 public class UserController {
8
9     @Autowired
10     private UserService service;
11
12     @Autowired
13     private AuthenticationManager authManager;
14
15     @Autowired
16     private JwtUtil jwtUtil;
17
18     @Autowired
19     private UserRepository userRepo;
20
21     // REGISTER API
22     @PostMapping("/register")
23     public ResponseEntity<User> register(@RequestBody User user) {
24         return ResponseEntity.ok(service.register(user));
25     }
26
27     // LOGIN API
28     @PostMapping("/login")
29     public ResponseEntity<User> login(@RequestBody User user) {
30         // Implementation for login
31     }
32 }
```

The console output shows the application startup logs, including the initialization of the HikariPool, JPA EntityManagerFactory, and the Spring application context.





Git Hub: https://github.com/Ujwal200707/FSAD_SKILL_EXPERIMENTS.git